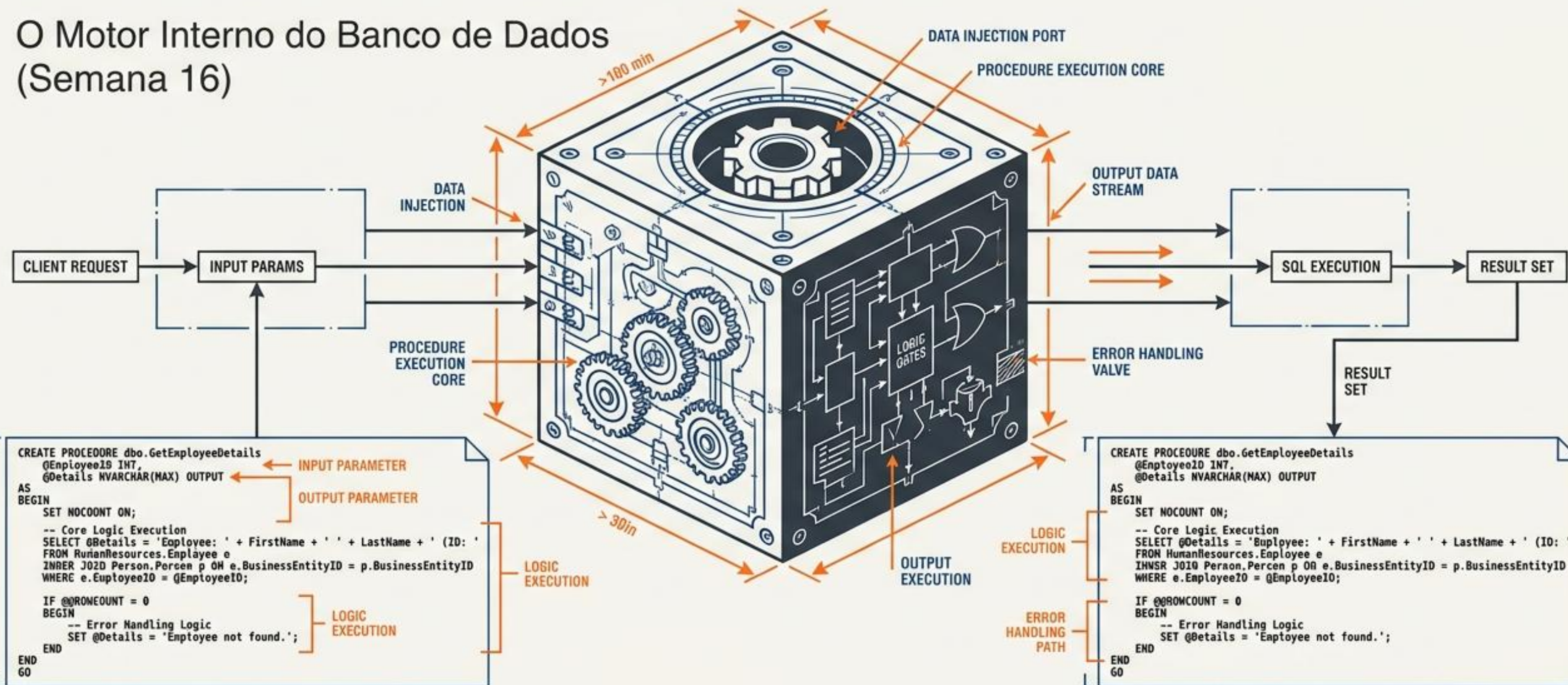


# Programação BD II: STORED PROCEDURES

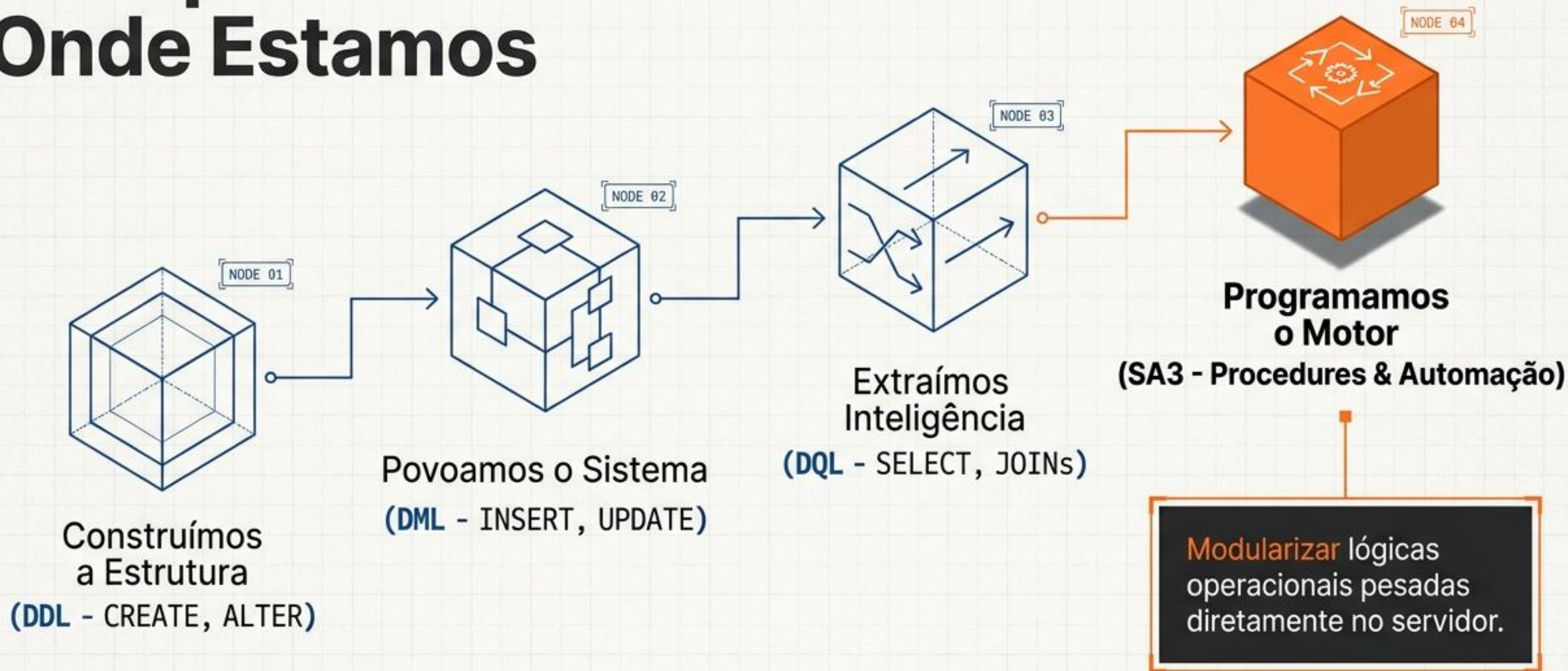
O Motor Interno do Banco de Dados  
(Semana 16)

Situação de Aprendizagem 3 (SA3)





# O Mapa da Jornada: Onde Estamos





# O Problema de Desempenho: O Tráfego na Rede



**C#/PHP**  
(Backend)

1. Backend envia SELECT para verificar o estoque. (Espera...)

2. Backend envia UPDATE para baixar o produto. (Espera...)

3. Backend envia INSERT para criar a fatura. (Espera...)

4. Backend envia INSERT para gerar o log. (Espera...)



**Banco de Dados**

**Múltiplas requisições** significam latência, gargalos e vulnerabilidade a falhas de conexão no meio da venda.



|             |  |          |
|-------------|--|----------|
| Projeto     |  | Rev. 001 |
| Descrição   |  |          |
| Responsável |  |          |

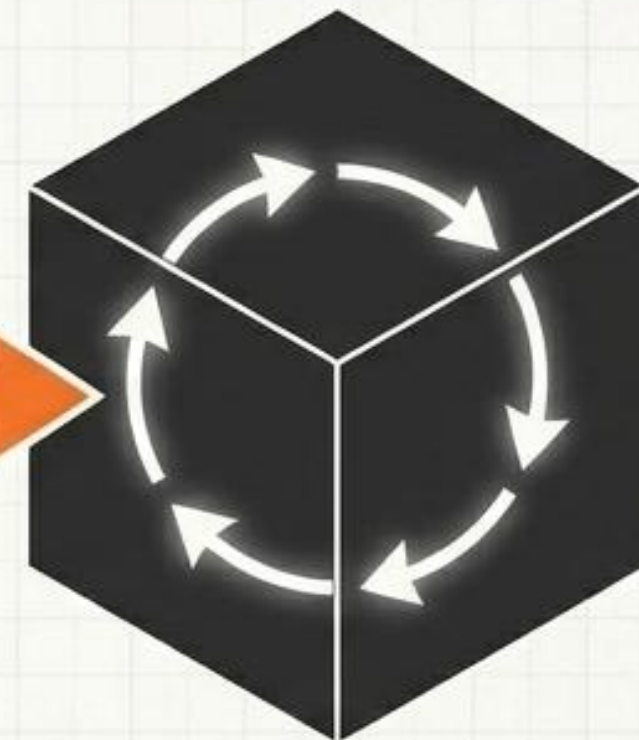


# A Solução: A Caixa Preta de Desempenho

O desenvolvedor envia apenas uma ordem.  
O Banco de Dados processa todos os passos internamente, em frações de segundo.



**C#/PHP**  
(Backend)



**Caixa Preta**

**Banco de Dados**



**Máxima Performance:**  
Zero tráfego de rede desnecessário.



**Segurança:**  
Blindagem do processo vital.



**Encapsulamento:**  
A regra de negócio mora no banco.





# A Anatomia do Motor: Sintaxe Básica

```
CREATE PROCEDURE sp_nome_da_rotina()  
BEGIN  
    -- Lógica SQL (INSERTs, UPDATEs, SELECTs)  
END
```

**CREATE PROCEDURE:**  
A declaração de  
nascimento do motor.

**BEGIN / END:** O  
envelope protetor. Tudo  
o que está aqui dentro  
pertence à rotina.



# O Guardião do Código: O Comando DELIMITER

## ☒ PROBLEMA



O MySQL usa o ; para executar comandos imediatos. Se usarmos ; dentro do **BEGIN/END**, o banco quebra a compilação pela metade.

## ☒ SOLUÇÃO



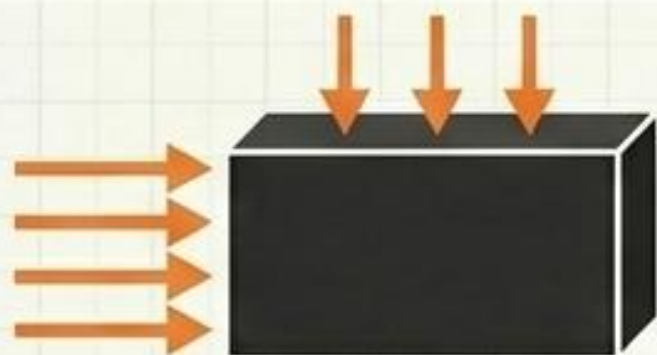
Trocamos **temporariamente** o finalizador padrão.

```
DELIMITER //  
CREATE PROCEDURE sp_exemplo()  
BEGIN  
    SELECT * FROM tabela;  
END //  
DELIMITER ;
```



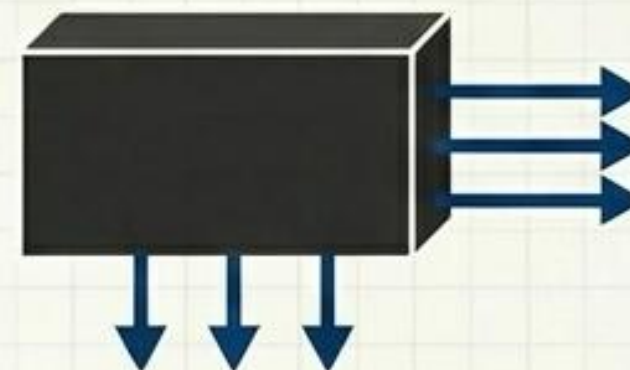
# Alimentando a Máquina: Parâmetros de Entrada e Saída

## Parâmetro IN



- A matéria-prima que o backend (PHP/C#) envia para o banco.
- **Analogia:** O dinheiro e os produtos entregues ao caixa.
- **Sintaxe:** IN p\_id\_cliente INT

## Parâmetro OUT



- O resultado processado que o banco devolve ao backend.
- **Analogia:** O recibo ou o troco saindo da máquina.
- **Sintaxe:** OUT p\_status VARCHAR(50)





# Ligando o Motor: O Comando CALL

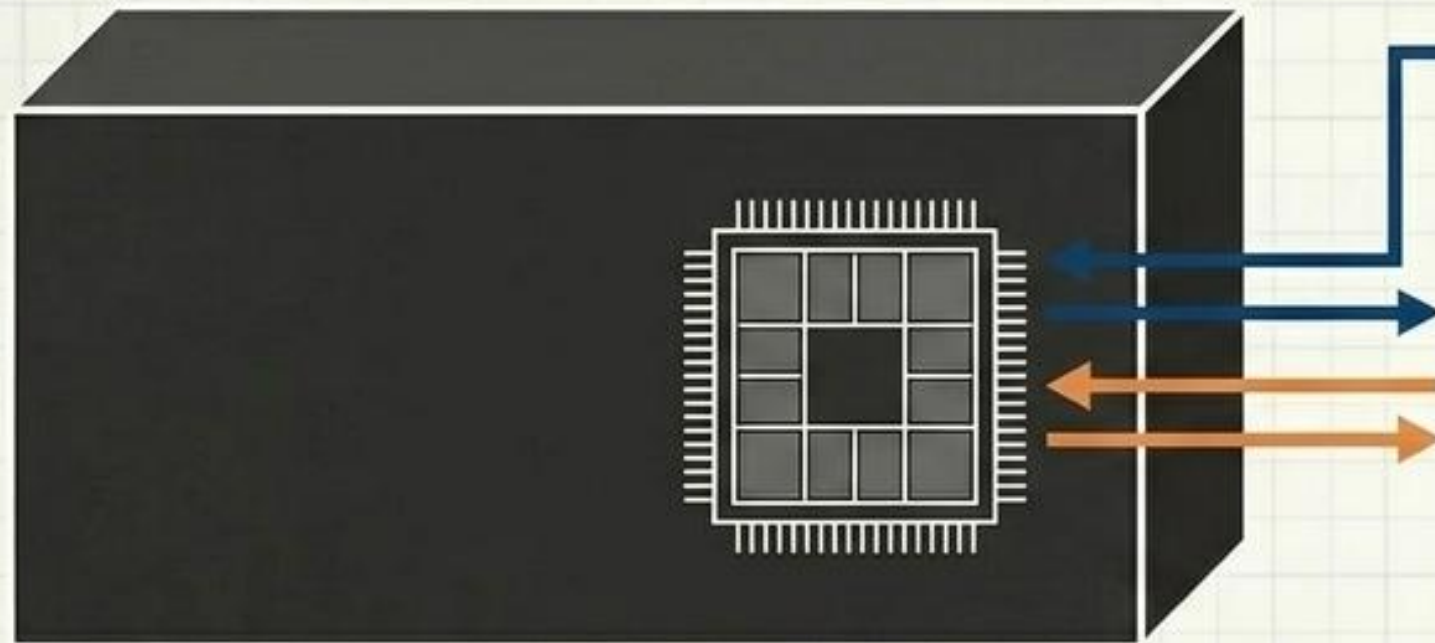
Criar a Procedure não a executa. Ela fica adormecida no servidor aguardando a invocação.

```
> CALL sp_nome_da_rotina('parametros', 'aqui');
```

**Um único CALL substitui dezenas de linhas de código soltas enviadas pela rede.**



# Memória Local: Variáveis de Estado (DECLARE)



Como a Procedure guarda dados temporários para 'pensar' antes de agir.

## Regras:

- Sempre declaradas imediatamente após o BEGIN.
- Exigem tipo de dado rigoroso.

```
DECLARE v_estoque_atual INT;  
DECLARE v_preco DECIMAL(10,2) DEFAULT 0.00;
```



# Mentoria do Arquiteto: Alerta de Escopo!

## O Perigo:

Nomear um parâmetro ou variável com o mesmo nome exato da coluna do banco.



## O Resultado:

Ambiguidade. O MySQL confunde quem é quem, e você pode atualizar a tabela inteira acidentalmente.

## O Padrão: Use prefixos!

|                         |                     |
|-------------------------|---------------------|
| O Padrão: Use prefixos! |                     |
| Parâmetros:             | <b>p_id_cliente</b> |
| Variáveis:              | <b>v_estoque</b>    |
| Colunas (Originais):    | <b>id_cliente</b>   |



# Inteligência e Decisão: IF / THEN / ELSE

Integrando lógica de programação estruturada diretamente no SQL.



```
IF v_estoque > 0 THEN
    -- Executa a venda (UPDATE / INSERT)
ELSE
    -- Aborta a operação e alerta o sistema
END IF;
```



# Atividade Guiada: O Primeiro Procedimento

**Missão:** Criar a sp\_dar\_desconto.

## Requisitos:

- Recebe um ID do produto como parâmetro (IN p\_id INT).
- Aplica um UPDATE reduzindo o preço atual em 10%.
- Uso correto de DELIMITER, BEGIN, e END.

>

**Execução:** Abra o ambiente de desenvolvimento. Preparem a sintaxe bruta.



# 0 Laboratório: Modularizando o Projeto 2

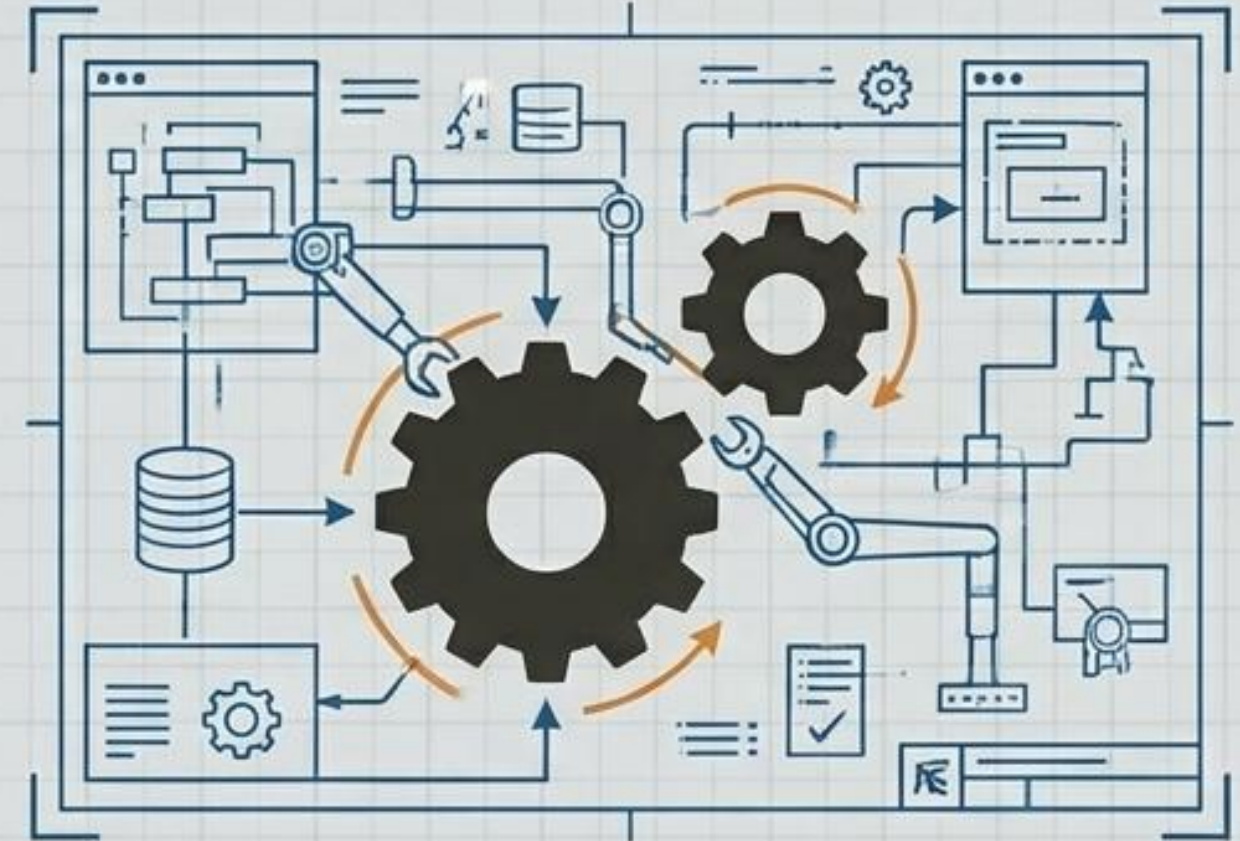
**0 Desafio:** O cliente exige que as operações vitais do sistema não dependam de comandos soltos no C#/PHP.

## Plano de Ação (Equipes):

1. Eleger as 2 operações mais complexas do sistema (Ex: Realizar Matrícula, Fechar Pedido).

2. Isolar a lógica pesada em Stored Procedures.

3. Testar a execução perfeita no console utilizando o CALL.





# O Escudo Arquitetônico contra SQL Injection



## A Ameaça

Sistemas mal projetados aceitam comandos soltos, permitindo que hackers injetem código malicioso pela tela de login ou URLs.

## A Defesa

A Procedure blinda o sistema. Ela aceita apenas o tipo de dado estrito parametrizado (ex: INT). Se o hacker enviar texto ou comandos soltos, o motor rejeita a entrada antes mesmo de processar o SQL.



# Elevando o Nível (Nível Autonomia - AUT)



**Contexto:** Para os esquadrões que finalizaram a modularização, o servidor **exige melhorias de resiliência e testes de stress.**

**Próximas Missões:** Tratamento de Exceções Críticas & Automação de Loops.



# Tarefa Extra 1: Tratamento de Exceções

## O Problema

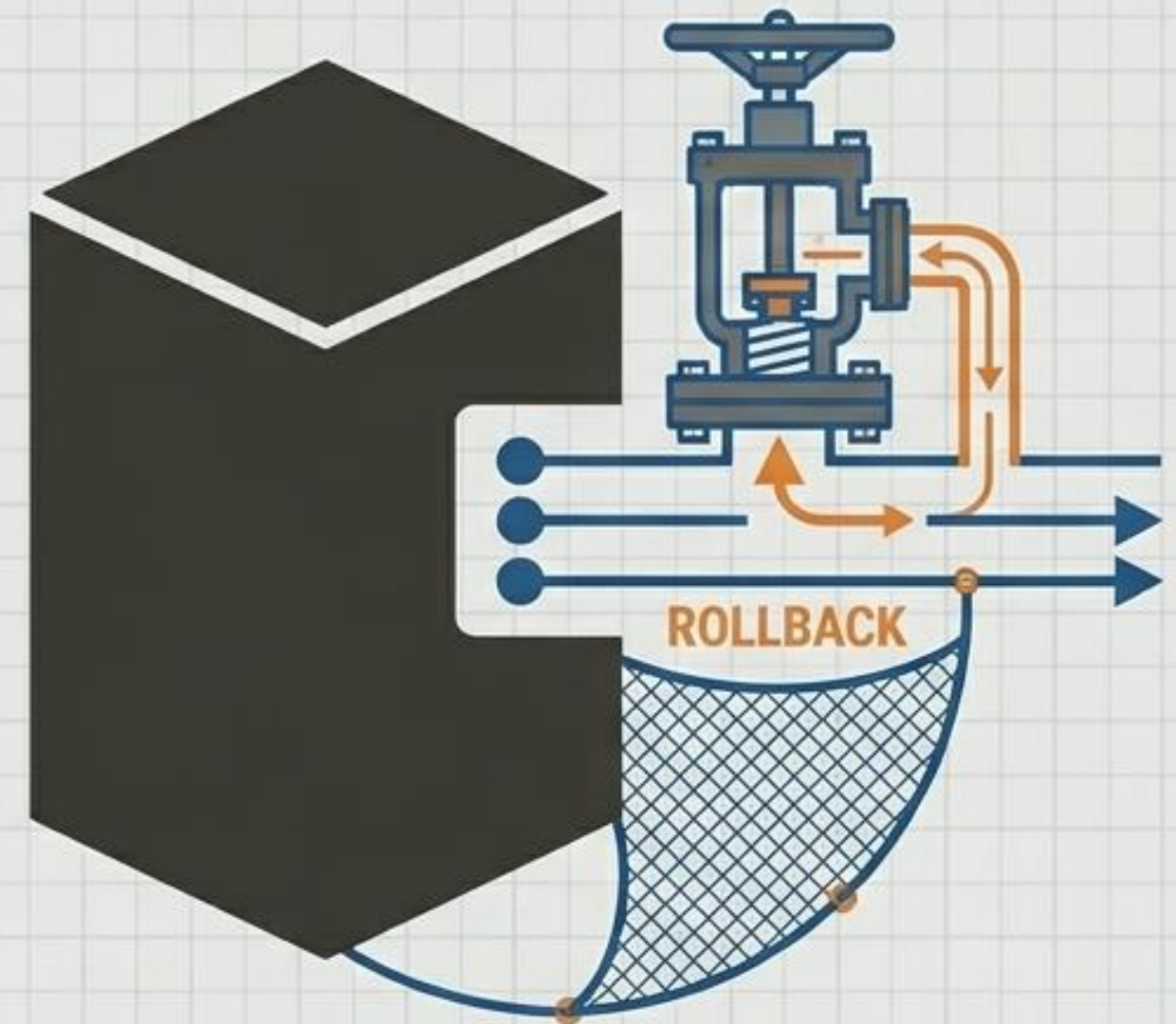
Se um erro fatal ocorrer no meio da Procedure, o sistema pode quebrar deixando dados inconsistentes.

## A Solução

Interceptar o erro e executar um ROLLBACK automático.

## Foco na Sintaxe:

```
DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
    ROLLBACK;
    SELECT 'Erro Crítico: Operação Abortada';
END;
```





# Tarefa Extra 2: O Loop Fabril (WHILE...DO)

## A Missão:

Como injetar uma massa gigante de dados para testes automatizados?

## O Conceito:

O banco de dados repetindo um **INSERT** milhares de vezes em frações de segundo com um único **CALL**.

## Estrutura do Código:

```
WHILE v_contador < 5000 DO  
    INSERT INTO tbl_testes (valor) VALUES (v_contador);  
    SET v_contador = v_contador + 1;  
END WHILE;
```





# Revisão Cruzada: Code Review

## Checklist de Auditoria (Validação entre as equipes):

- ☐ O **DELIMITER** foi alterado e restaurado corretamente?
- ☐ Os parâmetros (**IN/OUT**) possuem tipos de dados compatíveis?
- ☐ Não há colisão de escopo (prefixos **p\_** e **v\_** utilizados)?
- ☐ O comando **CALL** executa sem quebrar a integridade referencial?



# A Provocação: E se o Banco Reagisse Sozinho?

## A Limitação:

Hoje, criamos rotinas incríveis, blindadas e rápidas. Mas... elas ainda dependem de de um usuário ou sistema enviar o comando **CALL**.

A máquina não liga sozinha.

## A Pergunta:

E se o banco de dados funcionasse como um **alarme** de segurança inteligente, que dispara automaticamente quando cluir um dado crítico, sem ninguém precisar dar um **CALL**?

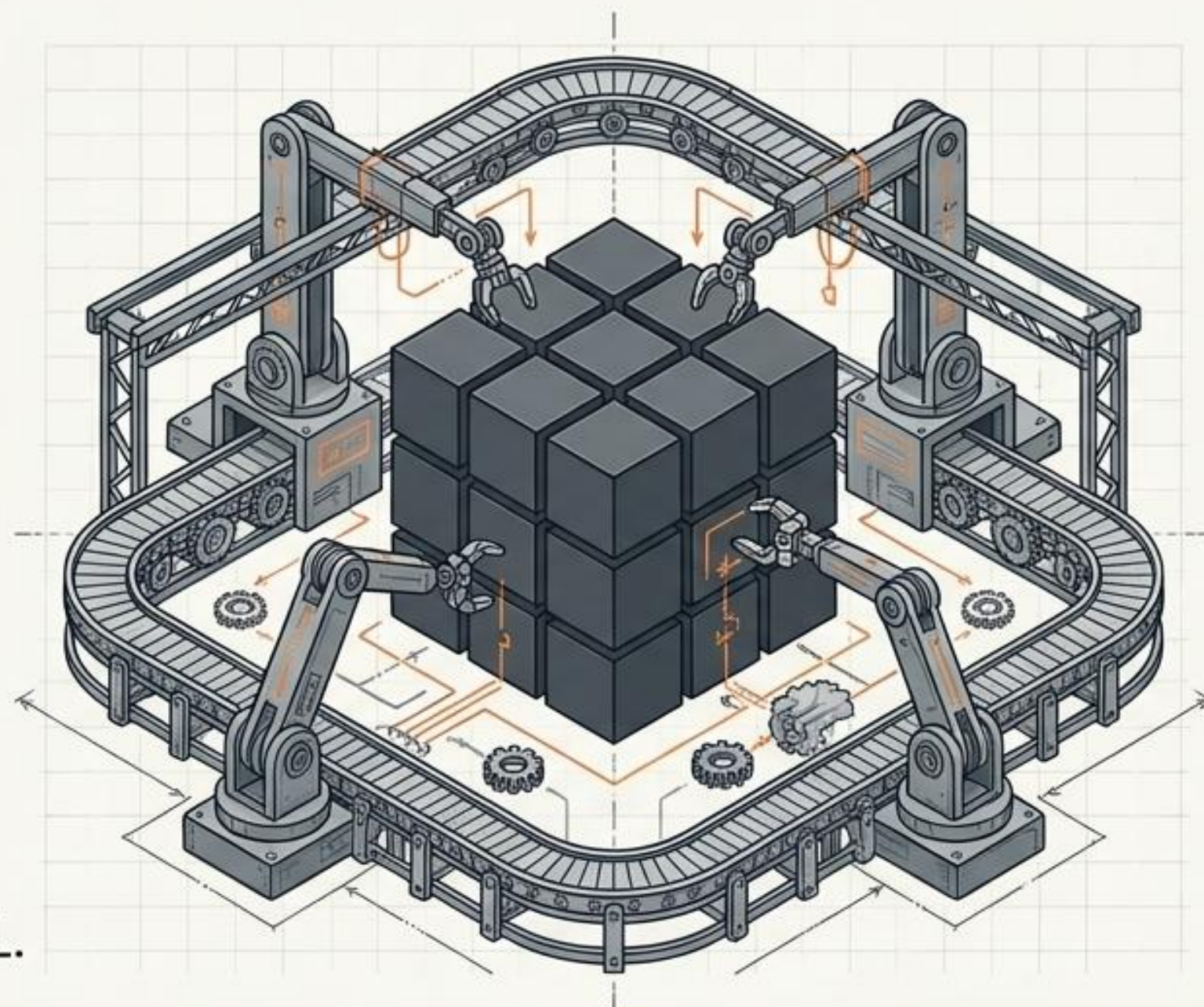




# Próximos Passos: O Servidor Autônomo

**Semana 17:**  
**FUNCTIONS &  
TRIGGERS**

**Foco: A Malha de  
Reação e Gatilhos.**  
Criação de auditoria  
passiva e rotinas que  
respondem  
sozinhas a eventos DML.



**Preparem-se para entregar a arquitetura definitiva do Projeto 2.  
O motor interno está oficialmente ligado.**